

Contrôle gestuel d'une application vidéo par vision artificielle

GOAREGUER Maël - 537407419
AKKAYA Garip - 537399808
BAUDART Alexandre - 537344920
POTIN Léa - 537396798

Date de remise : 11 décembre 2025

Cours : Vision numérique (GIF-7001)
Professeur : Robert Bergevin

Table des matières

1	Mise en contexte et description de la solution réalisée	3
1.1	Architecture technique	3
1.2	Fonctionnalités réalisées	3
1.3	Approfondissement technique	4
1.4	Scénario d'utilisation typique	4
1.5	Comparaison avec d'autres solutions	4
2	Justification des choix de design finaux	5
2.1	Choix technologiques	5
2.1.1	Python comme langage principal	5
2.1.2	PySide6 pour l'interface utilisateur	5
2.1.3	MediaPipe pour la détection de mains et la reconnaissance de gestes	5
2.1.4	OpenCV pour la capture vidéo et le traitement d'images	5
2.1.5	NumPy pour les calculs géométriques	5
2.2	Architecture modulaire	6
2.3	Configuration flexible	6
3	Résultats finaux obtenus	6
3.1	Détection de mains	6
3.2	Interface utilisateur	6
3.3	Reconnaissance de gestes	6
3.4	Contrôle vidéo	7
3.5	Infrastructure logicielle	7
3.6	Système de métriques et évaluation	7
4	Améliorations futures suggérées	8

1 Mise en contexte et description de la solution réalisée

L'interaction homme-machine évolue rapidement vers des interfaces plus naturelles et intuitives, dans le but de réduire la complexité d'utilisation et d'améliorer l'expérience utilisateur. Parmi ces interfaces, le contrôle gestuel s'impose comme une alternative prometteuse aux dispositifs classiques tels que la souris, le clavier ou la télécommande. Initialement popularisé par le domaine du jeu vidéo et les systèmes immersifs, le contrôle gestuel trouve aujourd'hui des applications dans la vidéo, la domotique, la santé, et l'industrie. En particulier, les lecteurs vidéo bénéficient directement de cette technologie, permettant à l'utilisateur de naviguer dans le contenu multimédia sans contact physique, de manière fluide et intuitive, et d'interagir tout en effectuant d'autres tâches. Ce type de contrôle offre non seulement une expérience plus immersive mais améliore également l'accessibilité pour les utilisateurs présentant des limitations motrices.

Le projet réalisé vise à développer un système de reconnaissance gestuelle capable de contrôler un lecteur vidéo via une webcam. L'objectif principal est de permettre à l'utilisateur d'interagir avec la vidéo par de simples mouvements de la main, sans nécessiter de contact physique avec le périphérique. Notre solution repose sur une architecture logicielle modulaire développée en Python, intégrant des technologies de vision par ordinateur et des outils spécialisés de détection de mouvements. L'architecture se compose de plusieurs modules principaux :

1.1 Architecture technique

- **Interface utilisateur** : Application PySide6 offrant une interface graphique claire et intuitive, intégrant un lecteur vidéo et des menus de configuration.
- **Capture vidéo** : Module OpenCV pour la capture et le prétraitement des images en temps réel (conversion en RGB, redimensionnement, réglage de la résolution et du taux de rafraîchissement).
- **Détection de mains** : Moteur MediaPipe Hands pour la détection et le suivi des mains, avec extraction de 21 landmarks par main et distinction entre les mains gauche et droite.
- **Traitement des gestes** : Système de reconnaissance de gestes basé sur des règles heuristiques, calculant distances et angles entre les articulations pour identifier les postures.
- **Contrôle de l'application** : Interface de communication avec le lecteur vidéo intégré, permettant la lecture/pause, l'avance ou le retour rapide, le réglage du volume et le mode plein écran.

1.2 Fonctionnalités réalisées

- Détection et suivi en temps réel des mains dans le flux vidéo.
- Reconnaissance de six gestes prédéfinis : “play/pause”, “avancer”, “reculer”, “volume up”, “volume down” et “fullscreen”.

- Interface utilisateur avec configuration des paramètres de détection et visualisation des landmarks en temps réel.
- Système de logging complet pour le débogage et l’optimisation.
- Architecture modulaire facilitant l’ajout de nouveaux gestes et fonctionnalités.
- Système de métriques permettant l’évaluation des performances et la création de matrices de confusion.

L’utilisation de MediaPipe présente plusieurs avantages : rapidité d’exécution garantissant un traitement en temps réel, précision élevée dans la détection grâce aux 21 points de repère par main, et intégration simplifiée via des modèles pré-entraînés. L’architecture modulaire assure une maintenance facile et permet l’évolution du système sans impacter les modules existants.



(a) Gestes de lecture (play/pause) et navigation (avancer/reculer)

(b) Gestes de volume (volume up/volume down) et plein écran (fullscreen)

FIGURE 1 – Illustration des gestes reconnus par le système

1.3 Approfondissement technique

Le module de capture vidéo utilise OpenCV pour acquérir les images à 30 FPS en résolution 1280×720 . Chaque image est prétraitée avant d’être transmise au moteur MediaPipe. Le moteur identifie les positions des mains et calcule les distances et angles entre les articulations pour déterminer la posture. Ces mesures servent ensuite au système de reconnaissance heuristique pour classifier les gestes. Un mécanisme de “debounce” empêche les déclenchements multiples lors de mouvements rapides ou tremblants. L’interface PySide6 visualise les landmarks et offre un retour en temps réel, permettant à l’utilisateur de confirmer visuellement que le geste a bien été reconnu.

1.4 Scénario d’utilisation typique

L’utilisateur lance l’application, active la capture vidéo, et interagit avec le lecteur. Pour mettre la vidéo en pause, il effectue un geste simple de la main. Le geste est détecté et confirmé par un retour visuel sur l’interface, et la vidéo se met instantanément en pause. De même, l’utilisateur peut avancer rapidement, reculer, ajuster le volume ou passer en plein écran uniquement avec des gestes naturels, sans toucher le clavier ou la souris.

1.5 Comparaison avec d’autres solutions

Contrairement aux systèmes utilisant des capteurs externes (Kinect, Leap Motion) ou des modèles d’apprentissage automatique complexes, notre solution purement logicielle

repose sur une webcam standard et MediaPipe. Cela la rend facilement déployable sur n'importe quel poste, avec une latence faible et une précision suffisante pour les commandes vidéo courantes, tout en restant extensible pour des fonctionnalités futures.

2 Justification des choix de design finaux

2.1 Choix technologiques

2.1.1 Python comme langage principal

Python a été choisi pour sa richesse en bibliothèques de vision par ordinateur et d'apprentissage automatique (TensorFlow, PyTorch, MediaPipe).

2.1.2 PySide6 pour l'interface utilisateur

PySide6 offre une interface multiplateforme, une intégration avec le système d'exploitation, et une excellente performance pour les applications de traitement vidéo en temps réel. PySide6 est utilisé pour créer l'interface graphique principale, gérer l'affichage vidéo, et implémenter les dialogues de métriques. Les signaux de Qt permettent une communication asynchrone entre les threads de capture vidéo et l'interface utilisateur, garantissant une réactivité optimale même lors du traitement intensif des frames vidéo en même temps que la webcam.

2.1.3 MediaPipe pour la détection de mains et la reconnaissance de gestes

MediaPipe Hands a été choisi pour sa capacité à détecter et suivre les mains en temps réel avec une latence faible. La bibliothèque fournit 21 points de repère (landmarks) par main, permettant une analyse précise de la posture des doigts. Ces landmarks sont utilisés pour calculer les distances et angles entre les articulations, formant la base d'un système de classification de gestes basé sur des règles heuristiques.

2.1.4 OpenCV pour la capture vidéo et le traitement d'images

OpenCV est utilisé pour la capture vidéo depuis la webcam via l'interface VideoCapture, permettant la configuration de la résolution et du taux de rafraîchissement. OpenCV fournit une interface robuste et optimisée pour la capture vidéo, avec support natif pour différentes sources (webcam, fichiers vidéo) et formats de résolution.

2.1.5 NumPy pour les calculs géométriques

NumPy est utilisé pour effectuer les calculs de distances euclidiennes entre les landmarks de MediaPipe, permettant de déterminer l'état des doigts (tendus/repliés) et de calculer les ratios de mouvement horizontal/vertical nécessaires à la classification des gestes.

2.2 Architecture modulaire

L'architecture modulaire adoptée sépare clairement les responsabilités :

- **Module video** : Gestion de la capture vidéo
- **Module engines** : Moteurs de traitement (MediaPipe)
- **Module processing** : Processeurs de flux vidéo
- **Module ui** : Interface utilisateur
- **Module utils** : Utilitaires (configuration, logging, métriques)

Cette séparation facilite les tests unitaires, la maintenance, et l'ajout de nouvelles fonctionnalités.

2.3 Configuration flexible

L'utilisation de fichiers YAML pour la configuration permet une personnalisation facile des paramètres sans modification du code source, facilitant l'adaptation à différents environnements et cas d'usage. Les paramètres configurables incluent les seuils de confiance de détection, les résolutions vidéo, et les paramètres de reconnaissance de gestes.

3 Résultats finaux obtenus

3.1 Détection de mains

Le système de détection a été implémenté et permet d'identifier les mains en temps réel dans le flux vidéo. Il affiche les 21 landmarks par main avec leurs connexions, traite les images à 30 FPS en résolution 1280×720 et gère différents seuils de détection et paramètres configurables. Le système supporte la détection de plusieurs mains simultanément et distingue correctement les mains gauche et droite.

3.2 Interface utilisateur

Une interface graphique a été développée pour visualiser le flux vidéo annoté en direct. Elle intègre des contrôles permettant d'activer ou désactiver la détection MediaPipe, un menu de paramètres pour ajuster la configuration, un visualiseur de journaux d'exécution et un système d'aide contextuelle. L'interface permet également de charger et contrôler des fichiers vidéo avec un lecteur intégré.

3.3 Reconnaissance de gestes

Un système de reconnaissance de gestes basé sur des règles heuristiques a été implémenté. Le système analyse les positions relatives des landmarks de MediaPipe pour reconnaître six gestes prédéfinis :

- **TOGGLE_PLAY_PAUSE** : Main plate avec tous les doigts tendus et collés
- **FULLSCREEN** : Main plate avec tous les doigts tendus et écartés

- **AVANCER** : Index pointé vers la gauche (mouvement horizontal)
- **RECULER** : Index pointé vers la droite (mouvement horizontal)
- **VOLUME_UP** : Index pointé vers le haut (mouvement vertical)
- **VOLUME_DOWN** : Index pointé vers le bas (mouvement vertical)

Le système utilise des seuils et ratios calculés à partir des distances entre landmarks pour différencier les gestes et éviter les confusions.

3.4 Contrôle vidéo

Le système permet de contrôler un lecteur vidéo intégré via les gestes détectés. Les commandes implémentées incluent la lecture/pause, l'avance rapide, le retour rapide, l'ajustement du volume en continu, et le passage en mode plein écran. Le système gère également les transitions de gestes pour éviter les déclenchements multiples grâce à un mécanisme de debounce.

3.5 Infrastructure logicielle

L'infrastructure complète du projet a été mise en place. L'application PySide6 assure une interface complète, appuyée par un système de capture vidéo multi-résolutions et l'intégration fonctionnelle du modèle MediaPipe pour la détection. L'architecture modulaire du code favorise la clarté et l'extensibilité, avec un système de configuration flexible basé sur YAML et un mécanisme de logging complet pour le débogage et le suivi.

3.6 Système de métriques et évaluation

Un système complet de métriques de performance a été implémenté pour évaluer la précision du système de reconnaissance de gestes. Ce système permet de :

- Mesurer les taux de détection de mains (vrais positifs, faux positifs, taux de détection)
- Évaluer la précision de reconnaissance de chaque geste via une matrice de confusion
- Exporter les résultats au format JSON, CSV ou image PNG

La matrice de confusion (voir Figure 2) compare les gestes déclarés manuellement par l'utilisateur via des touches clavier avec les gestes prédits par le système. Cette évaluation permet d'identifier les confusions entre gestes similaires et d'ajuster les seuils de détection en conséquence. Le système permet également de définir une vérité terrain pour la présence de mains, facilitant l'évaluation de la précision de détection.

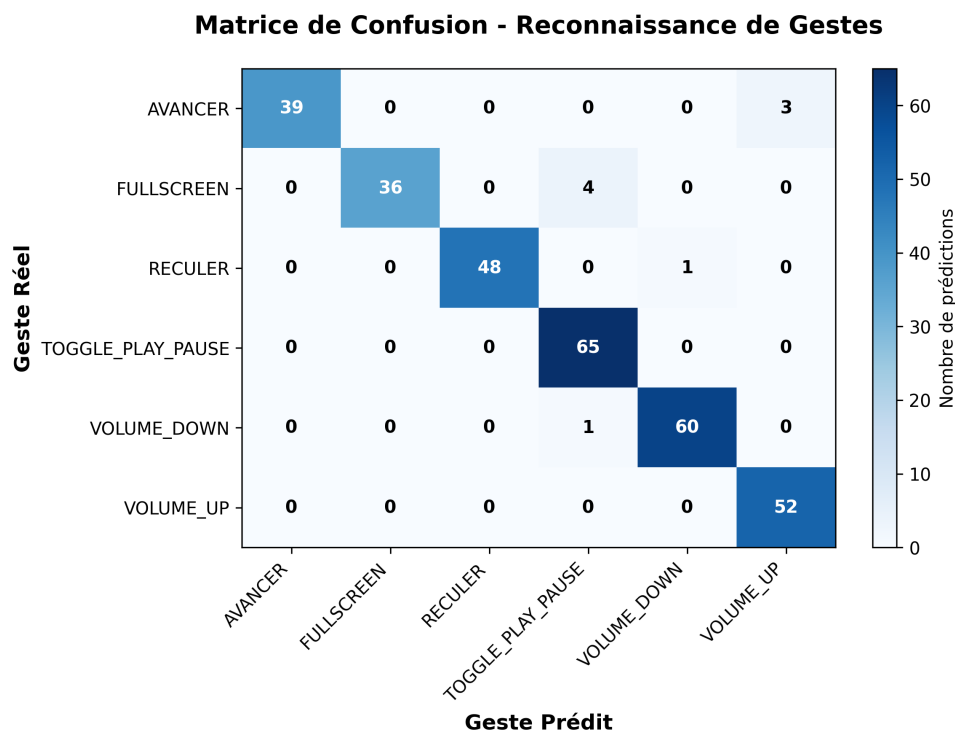


FIGURE 2 – Matrice de confusion des gestes reconnus

La matrice de confusion indique le nombre d’occurrences de chaque geste réel (lignes) prédit comme chaque geste (colonnes). La plupart des gestes sont correctement reconnus, avec de rares confusions : “Avancer” a été interprété 3 fois comme “Volume_Up”, et “Fullscreen” 4 fois comme “Reculer”. Les autres gestes présentent une reconnaissance quasi parfaite.

4 Améliorations futures suggérées

Plusieurs axes d’amélioration pourraient être explorés pour renforcer les performances et l’utilité du système :

- **Optimisation des seuils de détection** : Affiner davantage les seuils et ratios pour réduire les confusions entre gestes similaires, notamment entre les gestes de navigation (AVANCER/RECULER) et les gestes de volume (VOLUME_UP/VOLUME_DOWN).
- **Robustesse aux conditions d’éclairage** : Améliorer la détection dans différents environnements d’éclairage et distances de capture pour assurer une fiabilité optimale dans diverses conditions d’utilisation.
- **Approches alternatives de classification** : Explorer l’utilisation de modèles d’apprentissage automatique (réseaux de neurones) si les performances des règles heuristiques s’avèrent insuffisantes pour des gestes plus complexes.
- **Support multi-mains** : Étendre le système pour reconnaître des gestes impliquant les deux mains simultanément, permettant des commandes plus complexes et expressives.

-
- **Intégration avec des lecteurs vidéo externes** : Développer des interfaces de communication avec des lecteurs vidéo populaires (VLC, Windows Media Player) via des APIs ou des commandes système pour une compatibilité élargie.

Références

- MediaPipe Hands Documentation. <https://google.github.io/mediapipe/solutions/hands>
- OpenCV Documentation. <https://docs.opencv.org/>
- PySide6 Documentation. <https://doc.qt.io/qtforpython/>
- Lugaresi, C., et al. (2019). MediaPipe : A Framework for Building Perception Pipelines.